6주차 강의

제조데이터 분석과 최적화

제조데이터 표준 및 처리 방법

김한진
gks359@cbnu.ac.kr
충북대학교 산업인공지능연구센터

CHUNGBUK NATIONAL UNIVERSITY

INDAI INDUSTRIAL AI RESEARCH CENTER

INDUSTRIAL AI RESEARCH CENTER

CONTENTS

I 이미지 데이터를 활용한 AI 개발 - 크로메이트

II 이미지 데이터를 활용한 AI 개발 - MVTec AD

이미지 데이터를 활용한 AI 개발 - 크로메이트

INDUSTRIAL AI RESEARCH CENTER

■ 이미지 데이터를 활용한 AI 개발

➢ 크로메이트

목적 :

- ✓ 중크롬산염을 이용해 방청피막을 입히는 공정으로 광택이 필요한 하이엔드 제품에 사용되는 작업
- ✓ 피막을 형성하기 위한 주반응은 액 중의 산에 의해 아연이 용해하고, 중크롬산 이온 등을 환원하기 위해 아연계면에서는 수소 이온농도가 저하하고, 이 때문에 침전물이 아연 표면에 침작하고 복잡한 성분의 피막이 형성

[크로메이트 공정 전/후 제품 차이]

이미지 데이터를 활용한 AI 개발 - 크로메이트

INDUSTRIAL AI RESEARCH CENTER

■ 이미지 데이터를 활용한 AI 개발

➢ 제품 이미지 데이터

목적 :

✓ 데이터 분석을 통해 육안검사를 진행하던 다양한 업종의 품질검사 공정을 이미지 데이터 분석 기술을 통해 자동 검사를 적용

데이터 개수 :

✓ 전체 이미지 수: 1452개
✓ 정상 이미지 : 1378개
✓ 불량 이미지 : 74개

이미지 데이터를 활용한 AI 개발 - 크로메이트

INDUSTRIAL AI RESEARCH CENTER

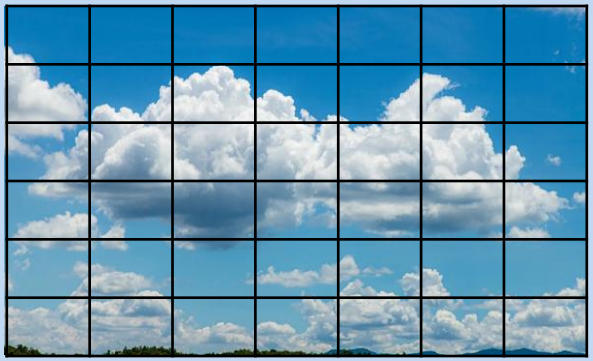
■ 왜 CNN이 필요한가?

1) 이미지 데이터의 특성

- 지역 상관성 (Local correlation) : 인접 픽셀들은 서로 비슷한 값을 갖는 경향이 큼
  → 하나의 픽셀은 독립적인 정보가 아니라 주변 픽셀들과 함께 어떤 의미 있는 패턴을 형성
- 지역적 균질성 : 이미지 전체는 비균질하지만, 작은 영역 단위로 보면 균질함
- 구성적/계층적 구조 : 엣지/코너 → 부분(눈,코) → 전체(얼굴)로 이어지는 계층적 표현

지역 상관성

- 같은 지역 내부에서는 픽셀들이 유사
- 경계 근처에서는 패턴 변화

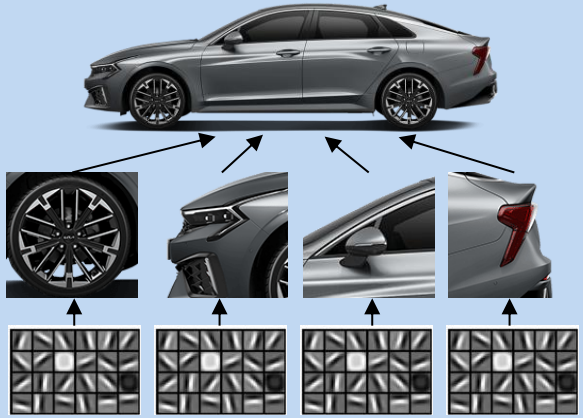


지역적 균질성

- 전체적으로는 균질해보이지 않음
- 작은 영역 단위로 보면 균질함

구성적/계층적 구조

- 이미지는 계층적으로 표현될 수 있음



■ 왜 CNN이 필요한가?

2) 기존 MLP의 한계

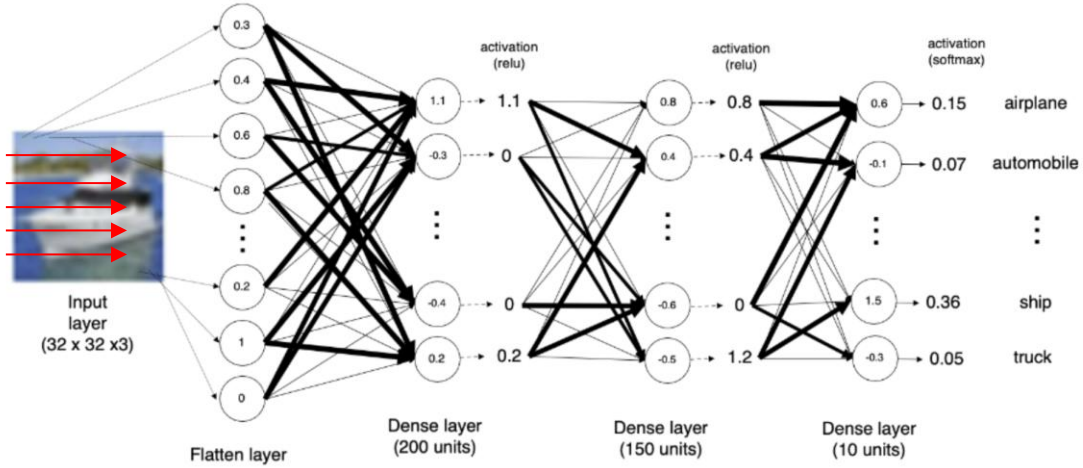
- 파라미터 폭발 : 입력 이미지의 크기에 따라 파라미터의 수가 폭발적으로 증가함

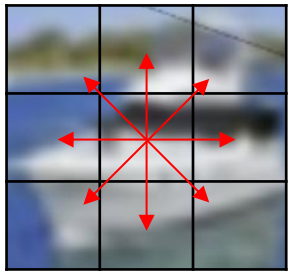
  ex) 224x224x3의 이미지를 100개 뉴런을 가진 MLP에 넣으면 150,528x100 → 15,052,800 (약 천만)

  ▶ 파라미터의 폭발적 증가는 일반화 성능 하락 (데이터 요구량 증가), 메모리 낭비

- 공간 구조 무시 : MLP는 이미지를 1열로 Flatten 시키므로 픽셀의 위치 정보 활용 불가

  ▶ 모든 픽셀을 독립적으로 취급, 픽셀의 이웃성 및 위치 구조 무시

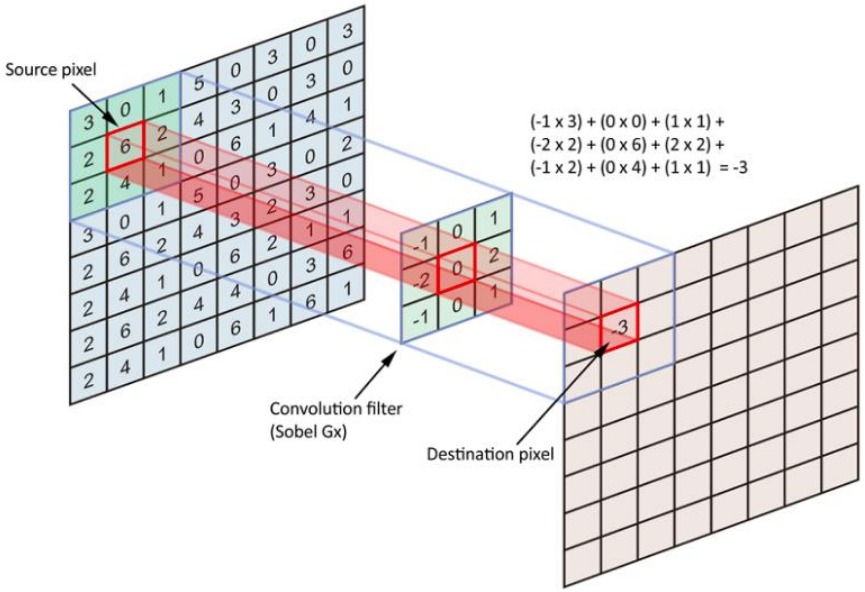


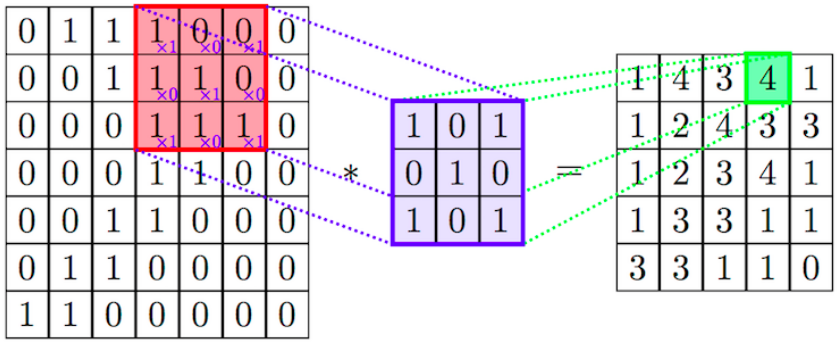


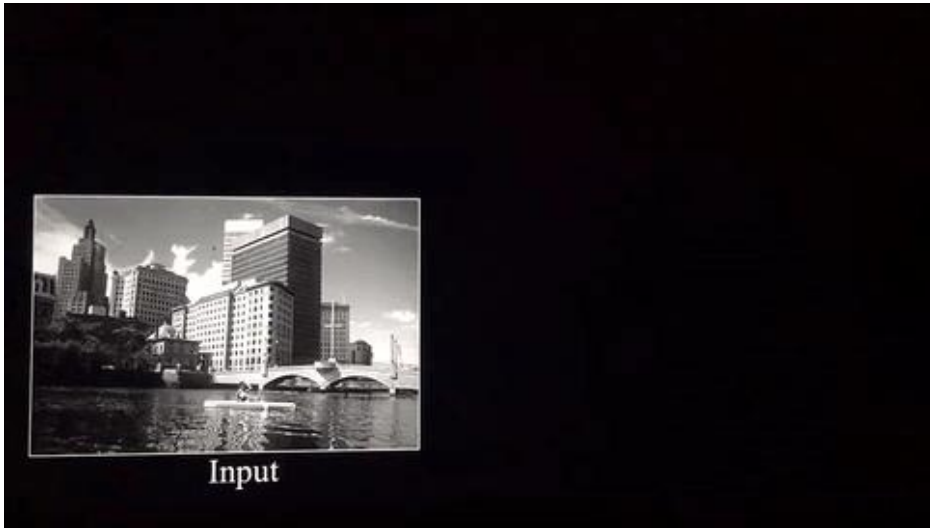
■ 컴퓨터 비전, 필터와 커널

- 과거 Computer Vision 에서는 이미지로부터 특징을 추출하기 위해 다양한 필터를 사용

컨볼루션 (Convolution) 연산

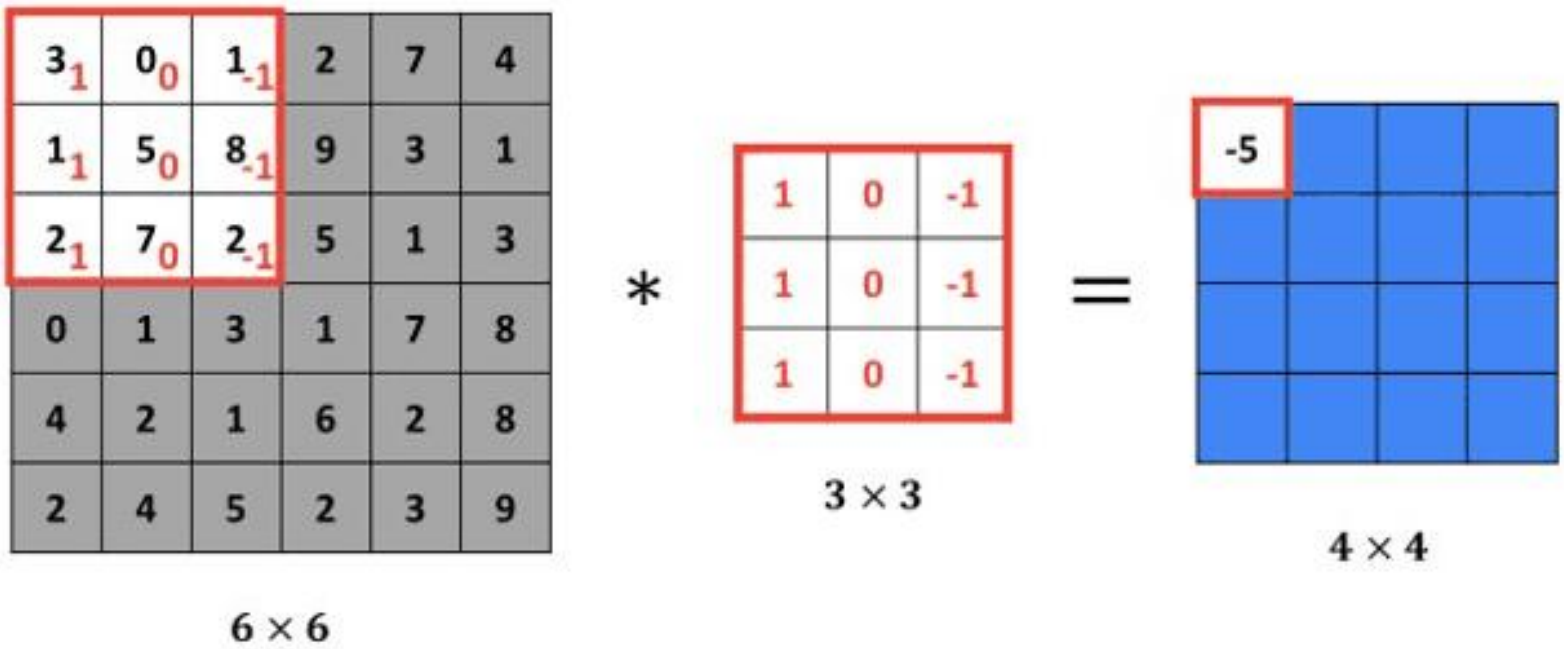






■ 컴퓨터 비전, 필터와 커널

Vertical Edge Filter, Convolution Example



$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

이미지 데이터를 활용한 AI 개발 - 크로메이트

INDUSTRIAL AI RESEARCH CENTER

CNN은 Convolutional Neural Network의
약자로,

출처 : https://www.youtube.com/watch?v=lDqn1UNwgrY

이미지 데이터를 활용한 AI 개발 - 크로메이트

INDUSTRIAL AI RESEARCH CENTER

■ 이미지 데이터를 활용한 AI 개발

➢ **step1_data_analysis.py**

인공지능 모델을 만들기 전, 위가 가진 데이터의 구성 확인
데이터셋에 나뉘어 있는 정상/불량 제품 이미지를 불러오고, 그 중 몇 장을 시각화

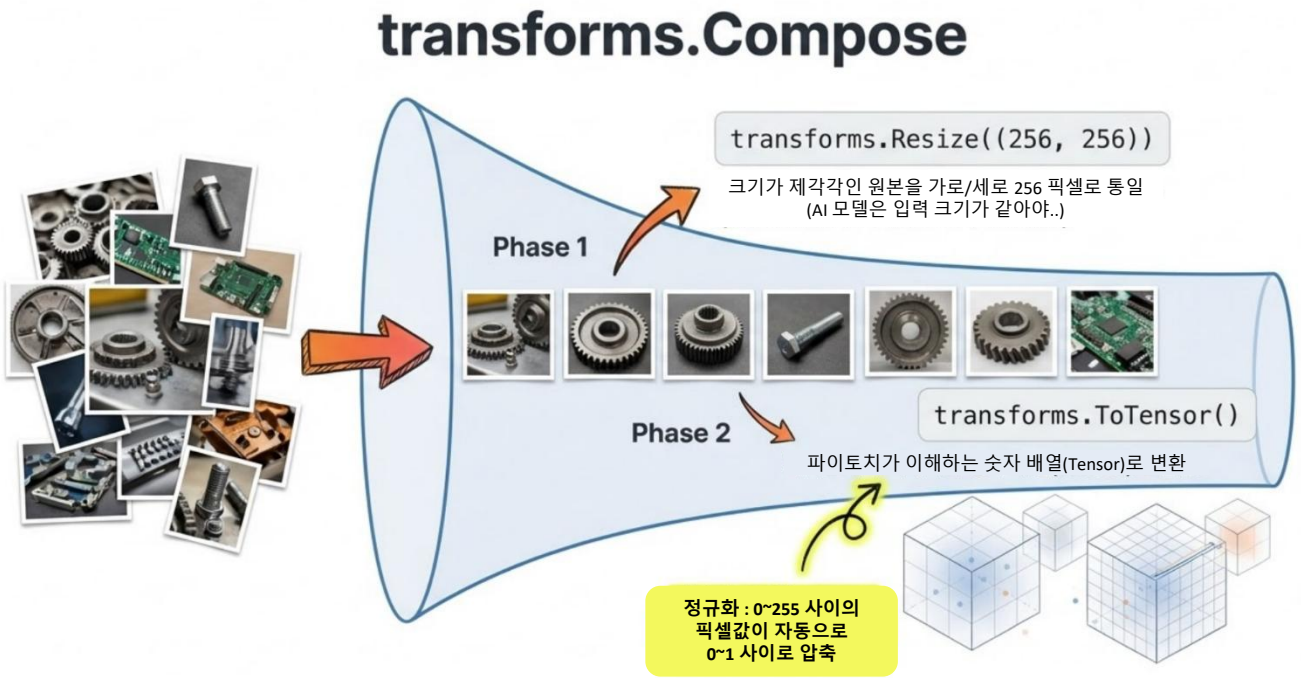
이미지 데이터를 활용한 AI 개발 - 크로메이트

INDUSTRIAL AI RESEARCH CENTER

■ 이미지 데이터를 활용한 AI 개발

➢ **step1_data_analysis.py**

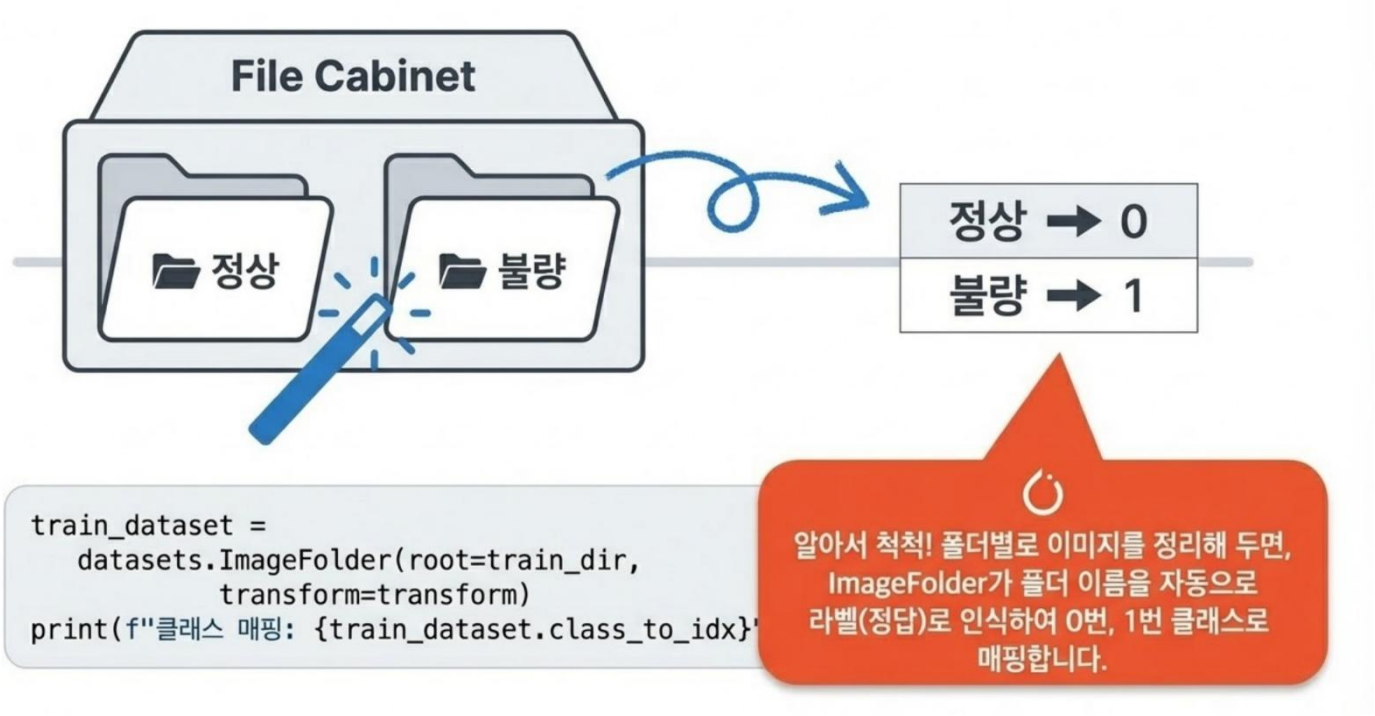
Transforms.compose는 여러 개의 이미지 변환 작업을 하나로 묶어주는 역할 수행



■ 이미지 데이터를 활용한 AI 개발

➢ **step1_data_analysis.py**

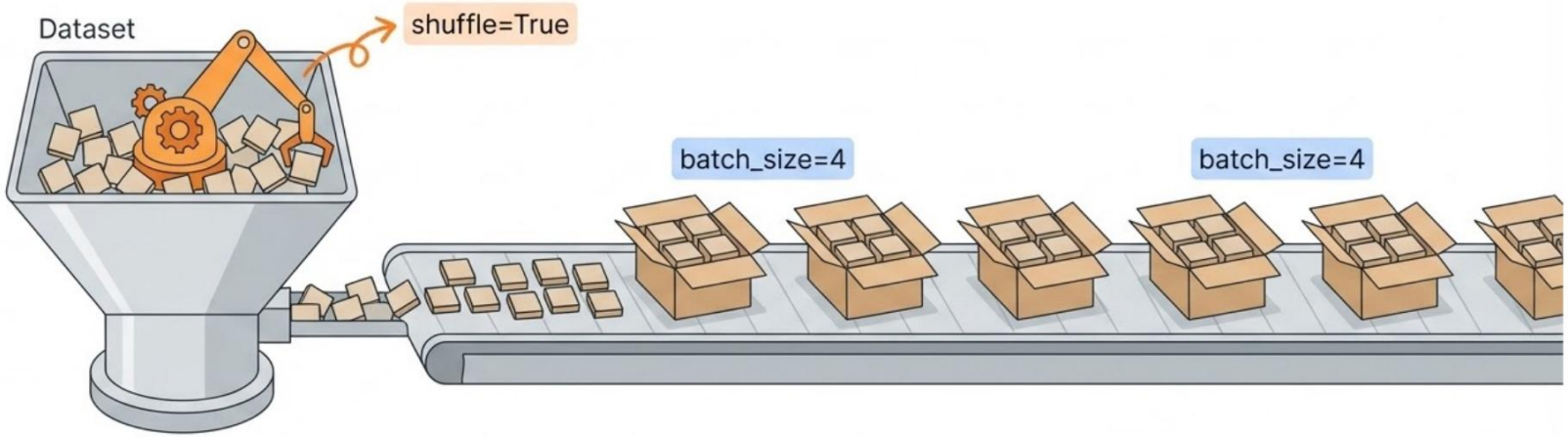
ImageFolder : Pytorch에서, 폴더 이름을 자동으로 라벨(정답)로 인식
예를 들면, 현재의 실습에서 '정상', '불량' 폴더가 있다면, 0번과 1번 클래스로 매핑



■ 이미지 데이터를 활용한 AI 개발

➢ **step1_data_analysis.py**

DataLoader : 방대한 데이터를 한 번에 모델에 넣을 수 없어, 설정한 batch_size 만큼 조각내어 가져옴




```
train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True)

images, labels = next(iter(train_loader))
```

방대한 데이터를 한 번에 넣을 수 없어 4개씩 조각내어 묶음(Batch) 생성

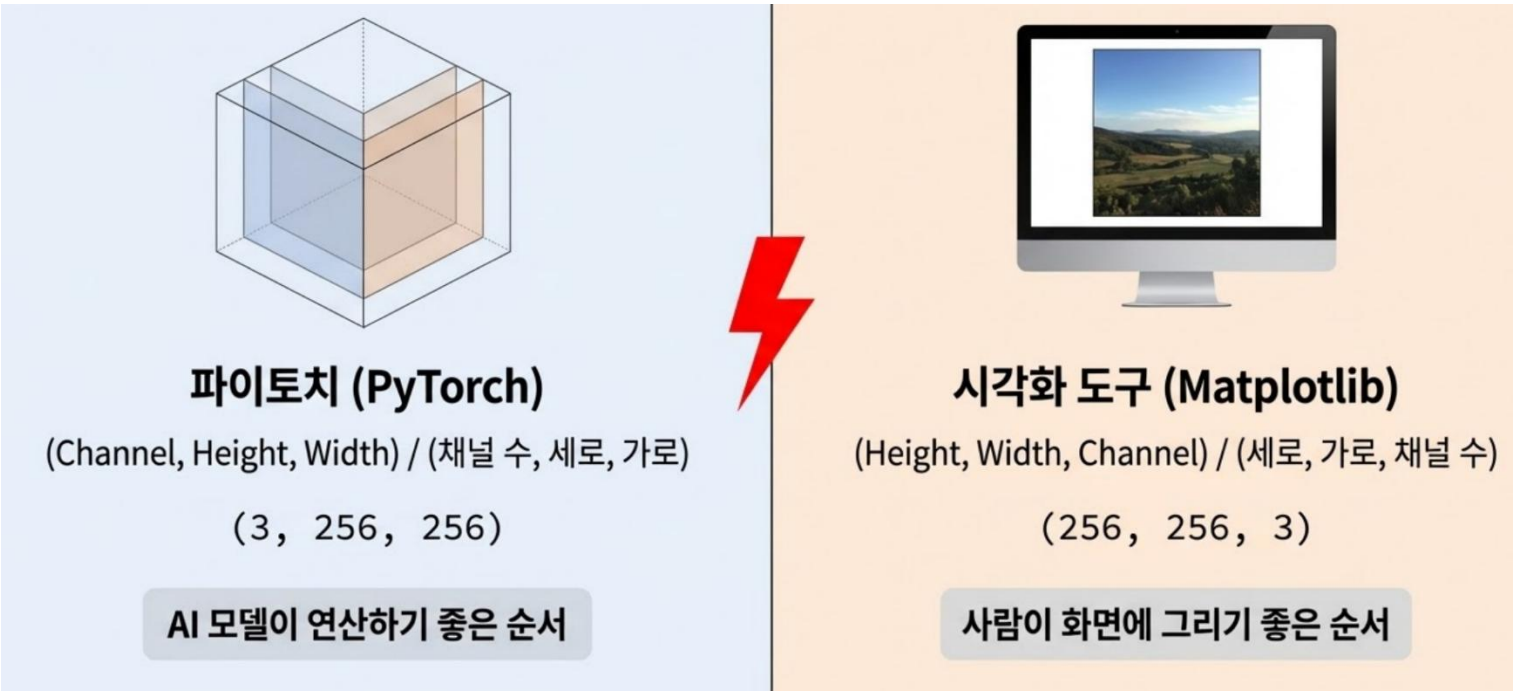
순서를 외우지 못하도록 shuffle

데이터 로더에서 첫 번째 묶음 (이미지 4장, 라벨 4개)을 한 번 꺼내옴

■ 이미지 데이터를 활용한 AI 개발

➢ **step1_data_analysis.py**

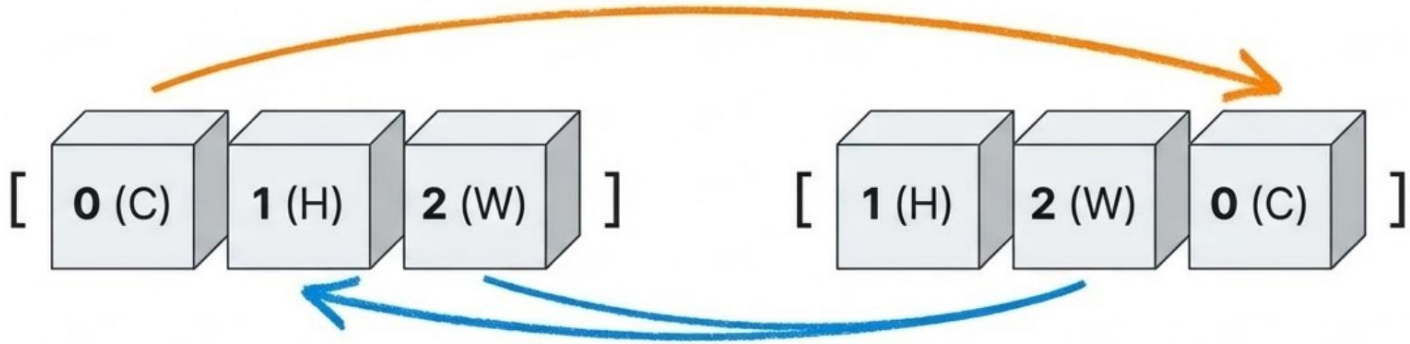
Matplotlib : 파이썬 라이브러리 중 하나로, 주로 데이터 시각화나 그래프를 그릴 때 사용



■ 이미지 데이터를 활용한 AI 개발

➢ **step1_data_analysis.py**

Permute(1,2,0) : pytorch의 1번(세로), 2번(가로), 0번(채널) 차원 순서로 축 위치 변경



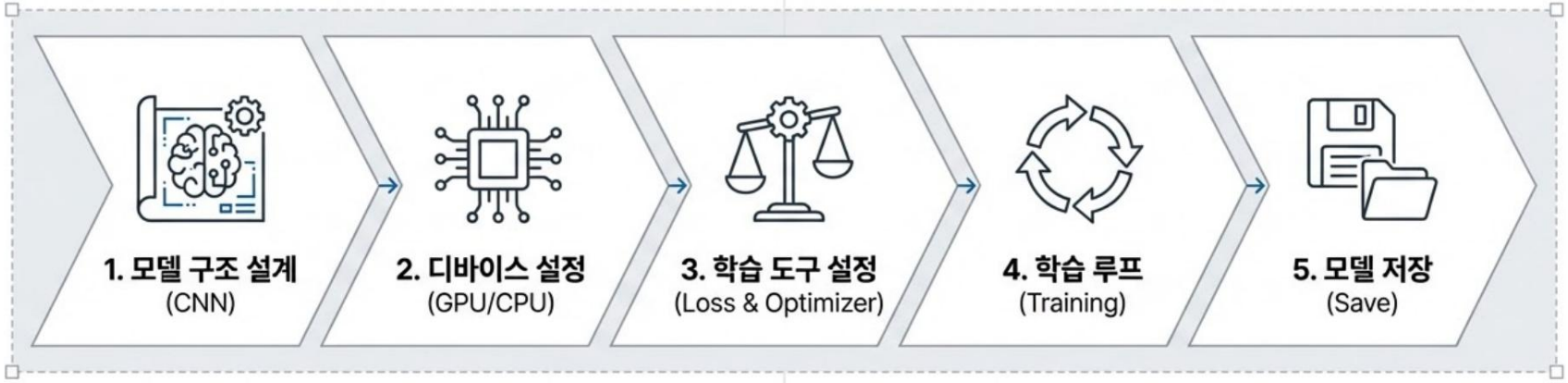

```
# PyTorch 텐서를 Matplotlib 형태로 변환
img = images[i].permute(1, 2, 0).numpy()
```

축의 위치를 강제로 섞어줍니다. 1번(세로)과 2번(가로)을 앞으로, 0번(채널)을 맨 뒤로 보냅니다.

파이토치 텐서를 시각화 도구가 읽을 읽을 수 있는 Numpy 배열로 최종 번역.

■ 이미지 데이터를 활용한 AI 개발

➢ **step2_train.py**
실제 인공지능 모델을 설계하고, 데이터셋을 활용하여 모델 훈련 진행



train.py는 데이터 세팅 이후, 실제 인공지능의 **'뇌(모델)'**를 **설계**하고
정답을 맞히도록 훈련시키는 **가장 핵심적인 단계**입니다.

이미지 데이터를 활용한 AI 개발 - 크로메이트

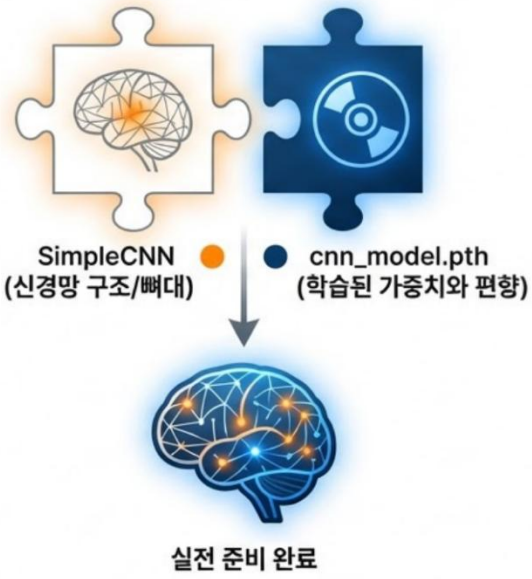
INDUSTRIAL AI RESEARCH CENTER

■ 이미지 데이터를 활용한 AI 개발

➢ **step2_train.py**
Pytorch를 활용하여 SimpleCNN 클래스 설계

```
from train import SimpleCNN

model = SimpleCNN().to(device)
model.load_state_dict(torch.load('cnn_model.pth'))
```



■ 이미지 데이터를 활용한 AI 개발

➢ step2_train.py
Pytorch를 활용하여 SimpleCNN 클래스 설계

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        # 특징 추출기 (Feature Extractor)
        self.conv_layers = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
```

- **nn.Module**: 파이토치에서 모든 신경망 모델이 기본적으로 상속받아야 하는 뼈대
- **nn.Sequential**: 여러 층(Layer)을 순서대로 묶어주는 컨테이너
- **nn.Conv2d (합성곱 층)**: 이미지에서 선, 색상, 질감 같은 **특징(Feature)을 추출**하는 역할
- **nn.ReLU (활성화 함수)**: 모델에 비선형성을 부여하여 복잡한 패턴을 학습
- **nn.MaxPool2d (풀링 층)**: 이미지의 크기를 절반으로 줄이면서 가장 중요한 특징만 남김 연산량을 줄이고 과적합을 방지하는 효과

■ 이미지 데이터를 활용한 AI 개발

➢ step2_train.py
Pytorch를 활용하여 SimpleCNN 클래스 설계

```
    # 분류기 (Classifier)
    self.fc_layers = nn.Sequential(
        nn.Flatten(),
        nn.Linear(64 * 64 * 64, 128),
        nn.ReLU(),
        nn.Dropout(0.5),
        nn.Linear(128, 2)
    )

def forward(self, x):
    x = self.conv_layers(x)
    x = self.fc_layers(x)
    return x
```

- **nn.Flatten()**: 2차원(면) 형태의 특징 맵을 1차원(선) 형태의 데이터로 길게 핌
  뒤에 나오는 Linear 층에 데이터를 넣기 위한 필수 작업
- **nn.Linear (완전 연결 층)**: 추출된 특징들을 바탕으로 최종적으로 이 이미지가 '정상'인지 '불량'인지 결정을 내리는 역할
  분류할 클래스가 2개(정상, 불량)이므로 출력층의 크기는 **2**
- **nn.Dropout(0.5)**: 학습 중 무작위로 뉴런의 50%를 끔
  모델이 특정 데이터에만 과도하게 익숙해지는
  과적합(Overfitting) 방지
- **forward 함수**: 데이터(이미지 x)가 모델에 들어왔을 때 어떤 순서로 층을 통과할지 정함

■ 이미지 데이터를 활용한 AI 개발

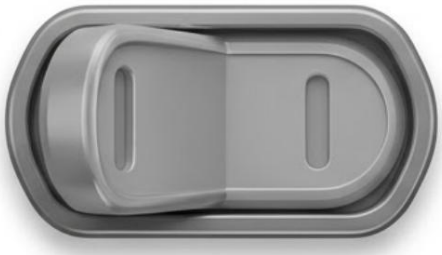
➢ **step3_evaluate.py**

모델이 학습 과정에서 본 적 없는 데이터, '테스트 데이터'를 활용하여 실제 시험 수행

```
model.eval()
```

'실전 시험 모드' ON

모델에게 실전 시험 중임을 알립니다.
과적합 방지를 위해 끄던 뉴런(Dropout 등)을
모두 활성화하여 100% 실력을 발휘합니다.



```
with torch.no_grad():
```

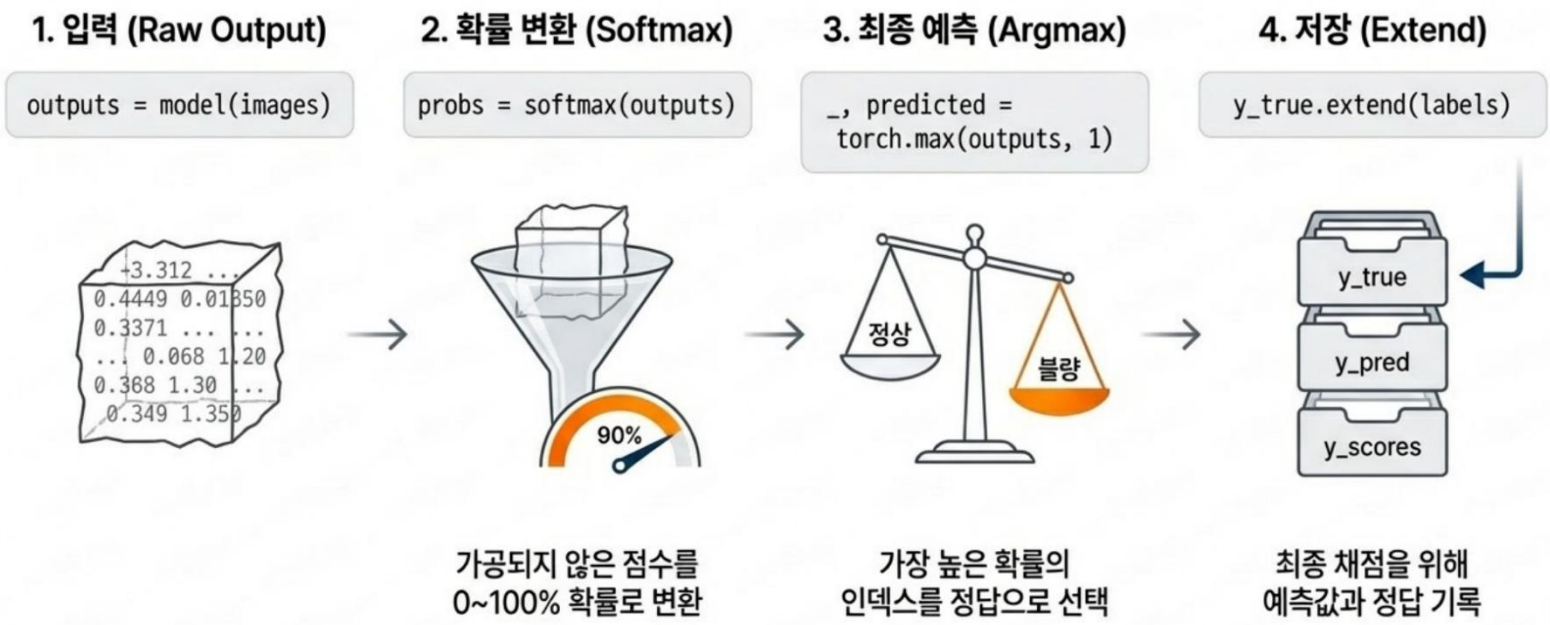
'오답노트(기울기 계산)' OFF

시험 중에는 가중치를 업데이트할 필요가 없습니다.
기울기 계산을 멈춰 메모리 사용량을 줄이고
평가 속도를 극대화합니다.

■ 이미지 데이터를 활용한 AI 개발

➢ **step3_evaluate.py**

Softmax : 모델이 출력한 가공되지 않은 점수(Logit)를 0~1 사이의 확률값으로 변환
Torch.max : 정상일 확률과 불량일 확률 중 더 높은 쪽의 인덱스를 최종 예측값으로 선택
리스트(extend) : 실제 정답, 모델의 예측값, 그리고 불량일 확률값을 추후 채점하기 위해 기록



INDUSTRIAL AI RESEARCH CENTER

■ 이미지 데이터를 활용한 AI 개발

➢ **step3_evaluate.py**

모델의 성능 평가 - 정확도, 정밀도, 재현율, F1-score 산출

Classification_report : Scikit-learn에서 제공하는 기능으로, 각 클래스별 상세한 성능 출력

```
print(classification_report(y_true, y_pred))
```

Accuracy (정확도)
전체 문제 중 몇 개를 맞혔는가?

Precision (정밀도)
모델이 '불량'이라 찍은 것 중, 진짜 '불량'의 비율
(오탐지 방지)

Recall (재현율)
실제 '불량'인 것 중, 모델이 놓치지 않고 찾아낸 비율
(미탐지 방지)

F1-Score
정밀도와 재현율의 조화 평균
(데이터 불균형 시 유용한 지표)

INDUSTRIAL AI RESEARCH CENTER

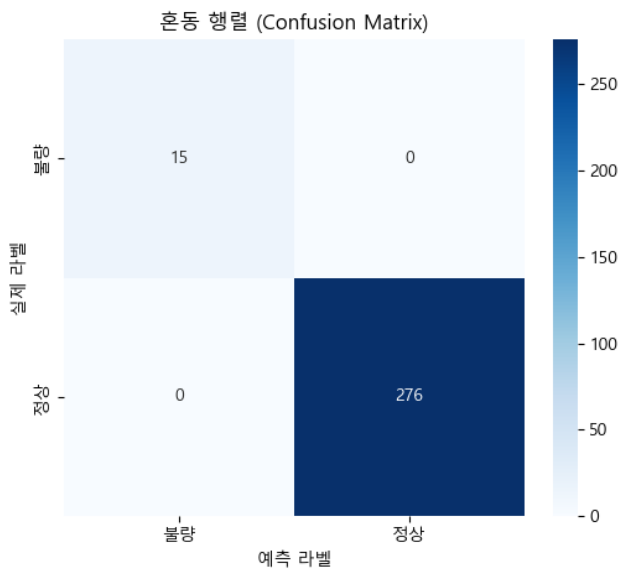
■ 이미지 데이터를 활용한 AI 개발

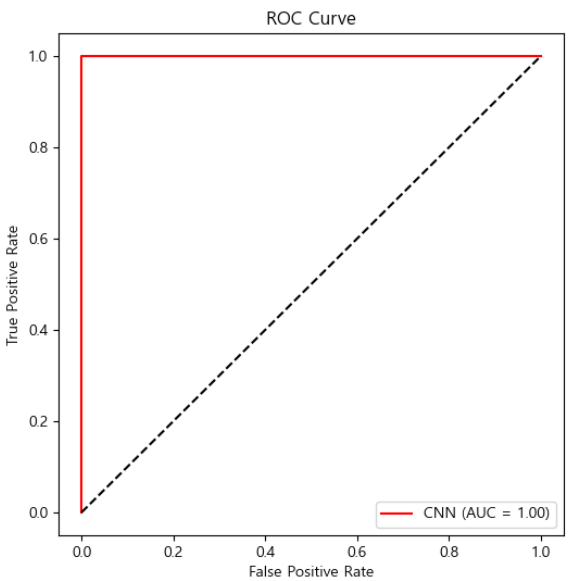
➢ step3_evaluate.py

Confusion matrix : 모델이 예측한 값과 실제 정답이 어떻게 매칭되는지 2x2 표 형태로 나타냄

ROC_curve : 분류 모델의 판단 기준이 변화함에 따라 모델 성능을 시각적으로 보여주는 그래프

AUC(Area Under the Curve) : ROC Curve 그래프 아래 면적





이미지 데이터를 활용한 AI 개발 - 크로메이트

INDUSTRIAL AI RESEARCH CENTER

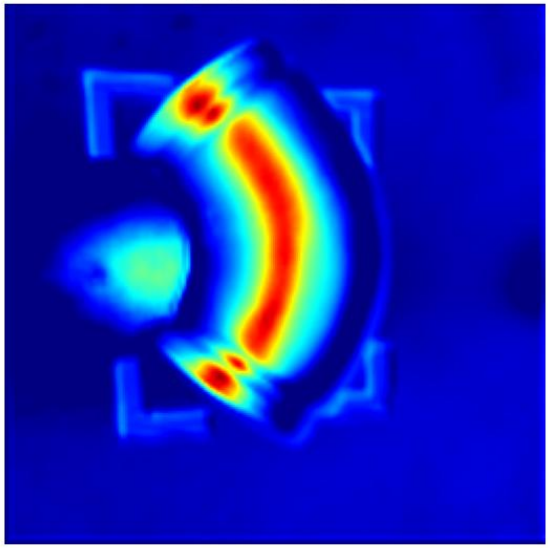
■ 이미지 데이터를 활용한 AI 개발

➢ **step4_inference.py**

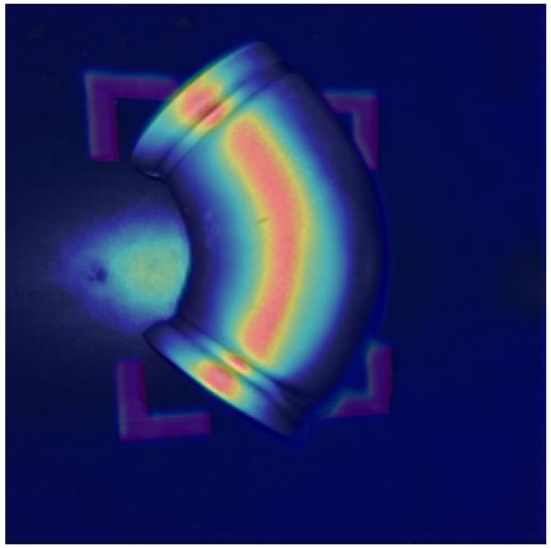
제품 이미지를 보고 "이 제품은 98%의 확률로 불량입니다!"라고 판정
Grad-CAM을 통해 AI가 이미지의 어느 부분을 보고 불량으로 판단했는지 추적

원본 제품 이미지

판단 주요 영역 (히트맵)



합성 결과: 불량 (98.9%)

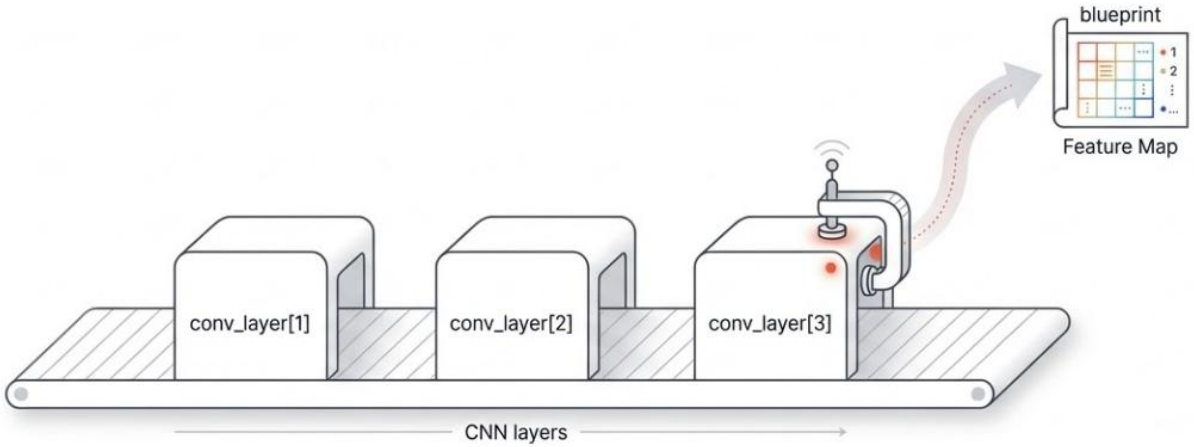


■ 이미지 데이터를 활용한 AI 개발

➢ step4_inference.py

register_forward_hook & backward_hook: CNN 모델의 특정 층(Layer)에 도청 장치(Hook)를 달아놓는 것

- **forward_hook:** 이미지가 층을 통과할 때 만들어지는 특징 맵(어떤 패턴을 찾았는지) 저장
- **backward_hook:** 정답을 역추적할 때 발생하는 기울기(어떤 패턴이 결정적인 역할을 했는지) 저장



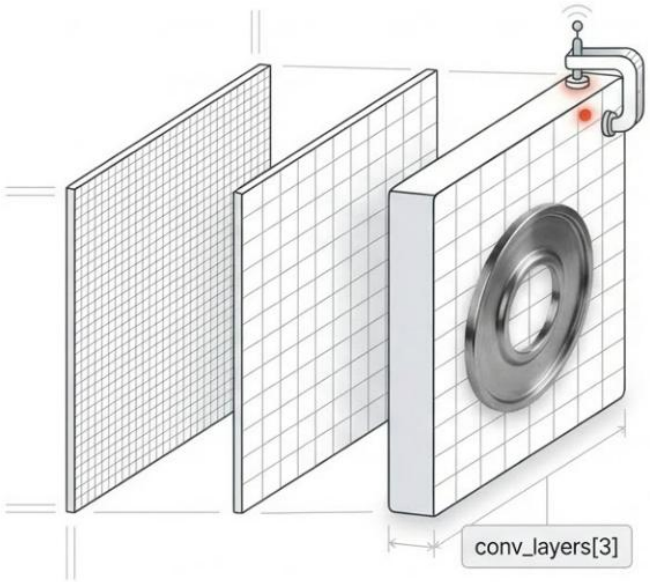
Hook(훅) 메커니즘: 모델의 내부 구조를 뜯어고치지 않고도, 특정 층(Layer)에 도청 장치를 달아 데이터가 통과할 때 그 정보(특징 맵, 기울기)를 몰래 복사해 저장합니다.

■ 이미지 데이터를 활용한 AI 개발

➢ step4_inference.py

register_forward_hook & backward_hook: CNN 모델의 특정 층(Layer)에 도청 장치(Hook)를 달아놓는 것

- **forward_hook:** 이미지가 층을 통과할 때 만들어지는 특징 맵(어떤 패턴을 찾았는지) 저장
- **backward_hook:** 정답을 역추적할 때 발생하는 기울기(어떤 패턴이 결정적인 역할을 했는지) 저장




```
target_layer = model.conv_layers[3]
cam_extractor = GradCAM(model, target_layer)
cam = cam_extractor(img_tensor, pred_idx)
```

왜 마지막 층인가?
모델의 가장 마지막 합성곱 층(Convolutional Layer)이 이미지의 전체적인 맥락을 **가장 잘 이해하고 있기 때문**입니다.

이미지 데이터를 활용한 AI 개발 - 크로메이트

INDUSTRIAL AI RESEARCH CENTER

■ 이미지 데이터를 활용한 AI 개발

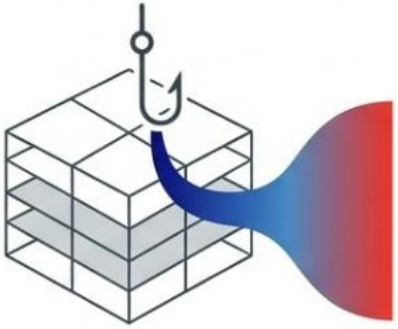
➢ **step4_inference.py**

OpenCV를 활용하여 추출된 시선(Grad-CAM) 데이터를 우리가 알아보기 쉽게 붉고 푸른 열화상 카메라(히트맵)처럼 색을 입혀 원본 사진과 겹침

입력 (Input)

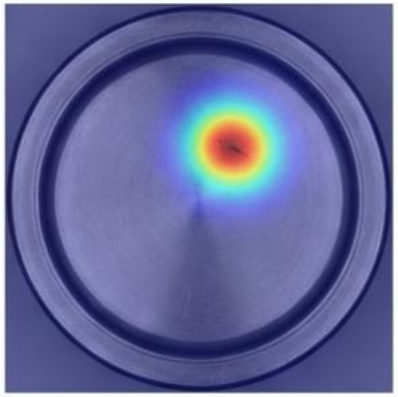
98% 불량 -> 질문: 왜? (Why?)

메커니즘 (Mechanism)



GradCAM + Hooks

출력 (Output)



설명 완료: 이 위치의 스크래치 패턴이 치명적입니다.

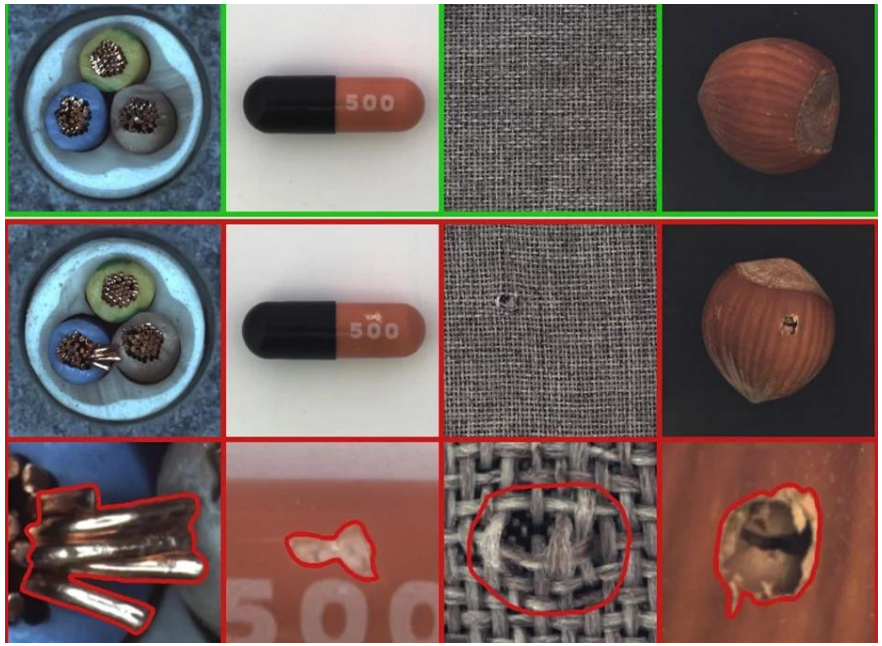
이미지 데이터를 활용한 AI 개발 – MVTec AD

INDUSTRIAL AI RESEARCH CENTER

■ 이미지 데이터를 활용한 AI 개발

➢ **MVTec AD (anomaly Detection) 데이터셋**

- 독일의 MVTec Software GmbH에서 발표한 산업용 이미지 이상 탐지 전용 데이터셋
- 실제 제조 현장에서 발생할 수 있는 다양한 결함 상황을 고해상도 이미지로 정밀하게 재현하여, 비지도 학습(Unsupervised Learning) 기반의 알고리즘 성능을 평가하는 데 최적화

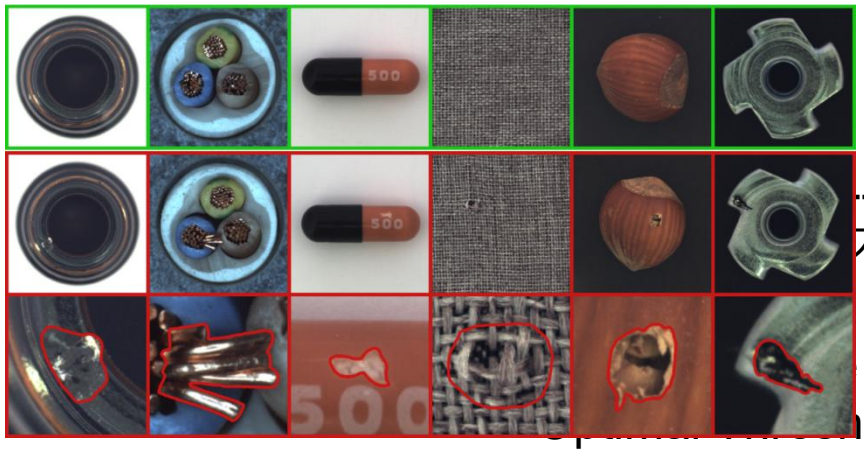


INDUSTRIAL AI RESEARCH CENTER

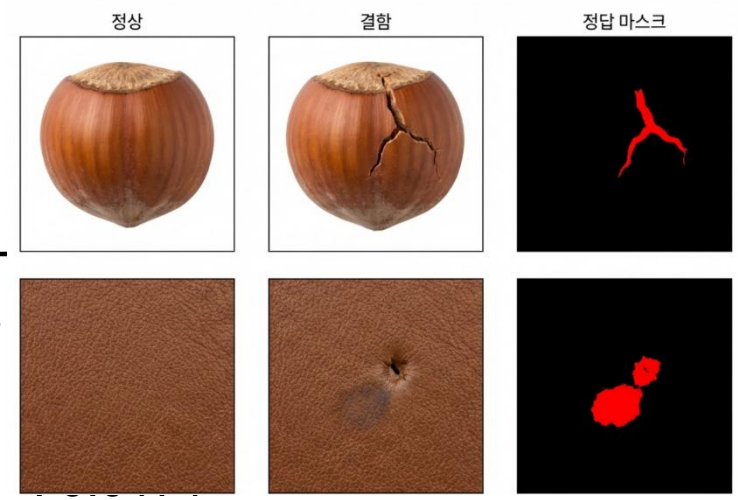
■ 이미지 데이터를 활용한 AI 개발

➢ MVTec AD (anomaly Detection) 데이터셋

- **객체(10종) :** 병(Bottle), 케이블(Cable), 캡슐(Capsule), 헤이즐넛(Hazelnut), 금속 너트(Metal Nut), 알약(Pill), 나사(Screw), 칫솔(Toothbrush), 트랜지스터(Transistor), 지퍼(Zipper) → 구조적 변형이나 부품 누락 등 탐지
- **질감(5종)** : 카펫(Carpet), 격자(Grid), 가죽(Leather), 타일(Tile), 나무(Wood) → 패턴이 깨지거나 오염 탐지



MVTec AD 데이터셋 샘플



정상 vs 결함 비교

이미지 데이터를 활용한 AI 개발 – MVTec AD

INDUSTRIAL AI RESEARCH CENTER

■ 이미지 데이터를 활용한 AI 개발

➢ MVTec AD (anomaly Detection) 데이터셋

① 비지도 학습(Unsupervised) 구조

- **Train Set:** 오직 결함이 없는 **'정상(Good)' 이미지**로만 구성
  모델이 정상 상태를 완벽히 학습하도록 유도
- **Test Set:** 정상 이미지와 함께 **다양한 종류의 불량 이미지**가 포함되어 있어, 모델이 본 적 없는 패턴(이상치)을 얼마나 잘 구분하는지 테스트

② 픽셀 단위의 정답지 (Ground Truth)

- 단순히 "이 이미지는 불량이다"라는 라벨링을 넘어, **결함의 정확한 위치와 모양을 나타내는 픽셀 단위 마스크(Mask)** 데이터 제공
- 모델이 결함 부위를 얼마나 정확하게 짚어냈는지(Segmentation 성능) 정밀하게 평가

③ 실제 산업 현장의 결함 반영

- 인위적인 노이즈가 아니라, 실제 공정에서 흔히 발생하는 **스크래치(Scratch), 오염(Contamination), 찌그러짐(Dent), 부러짐(Broken)** 등 수십 가지의 실제 결함 유형 포함

이미지 데이터를 활용한 AI 개발 – MVTec AD

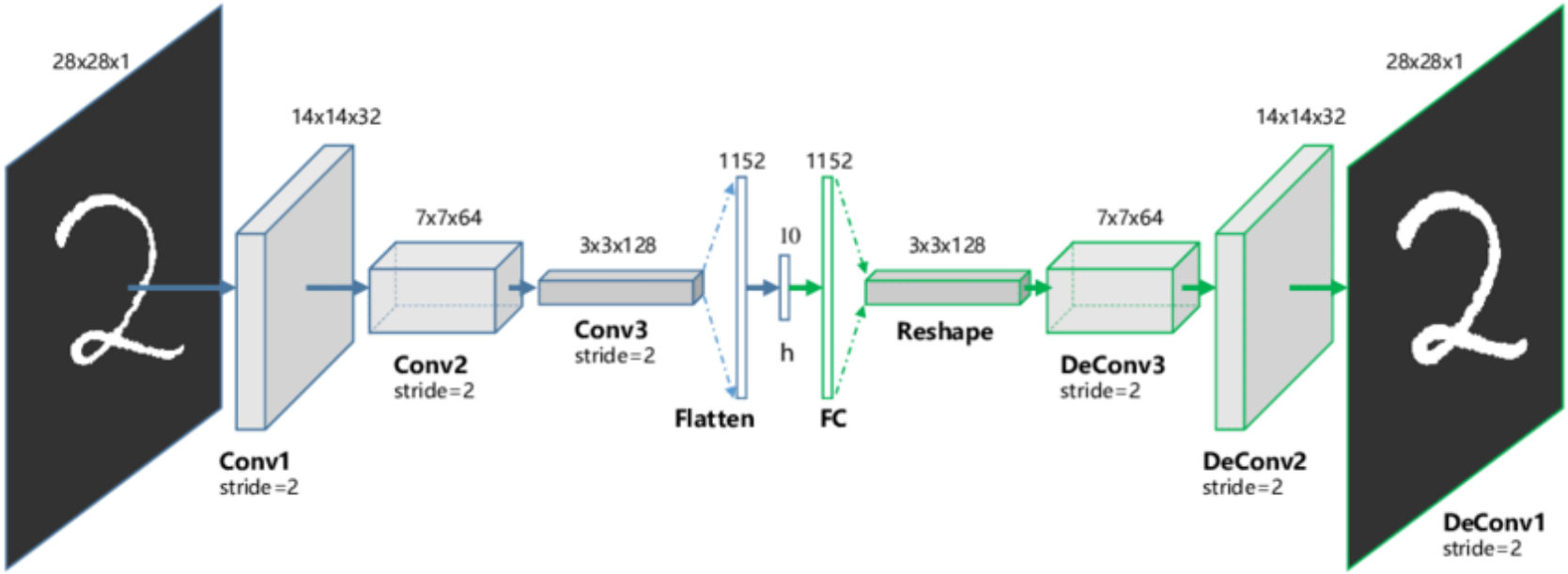
INDUSTRIAL AI RESEARCH CENTER

■ 이미지 데이터를 활용한 AI 개발

➢ 합성곱 오토인코더(Convolutional Autoencoder, CAE)

이미지 데이터의 차원을 축소하고 핵심적인 시각적 특징을 추출하는 데 특화된 비지도 학습 모델

일반적인 오토인코더의 데이터 압축 및 복원 구조에 CNN의 공간적 특징 추출 능력 결합



이미지 데이터를 활용한 AI 개발 – MVTec AD

INDUSTRIAL AI RESEARCH CENTER

■ 이미지 데이터를 활용한 AI 개발

➢ 합성곱 오토인코더(Convolutional Autoencoder, CAE)

- **인코더 (Encoder):** 입력된 이미지의 공간적 차원(가로, 세로)을 줄이면서 채널 차원(특징 맵)을 늘려가는 과정
  합성곱 층(Convolutional Layer)과 풀링 층(Pooling Layer)이 반복적으로 사용됨
  → 네트워크는 이미지의 국소적인 특징(선, 모서리 등)부터 전역적인 특징(형태, 질감 등)까지 계층적으로 파악

- **잠재 공간 (Latent Space / Bottleneck):** 인코더를 거쳐 고차원의 원본 데이터가 저차원으로 압축된 상태
  잠재 벡터(Latent Vector)는 원본 이미지의 가장 필수적이고 압축된 정보만을 담고 있는 모델의 '핵심 요약본' 역할

- **디코더 (Decoder):** 압축된 잠재 벡터를 다시 원본 이미지와 같은 크기와 형태로 복원
  이를 위해 업샘플링(Upsampling) 층이나 전치 합성곱(Transposed Convolution, 때로는 Deconvolution이라 불림) 층을 사용하여 점진적으로 공간적 차원을 늘림

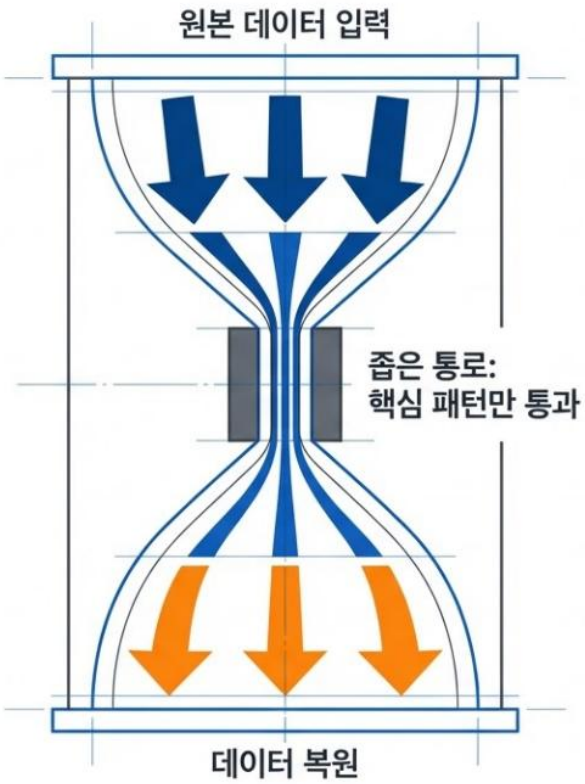
기존의 완전 연결 층(Fully Connected Layer)만으로 구성된
오토인코더는 이미지를 1차원 벡터로 길게 펼쳐서 연산
→ 이 과정에서 픽셀 간의 2차원적인 공간적 상관관계(Spatial Correlation)가 깨짐

이미지 데이터를 활용한 AI 개발 – MVTec AD

INDUSTRIAL AI RESEARCH CENTER

■ 이미지 데이터를 활용한 AI 개발

➢ 합성곱 오토인코더(Convolutional Autoencoder, CAE)



"복사기"가 아닌 "모래시계": 정보의 취사선택

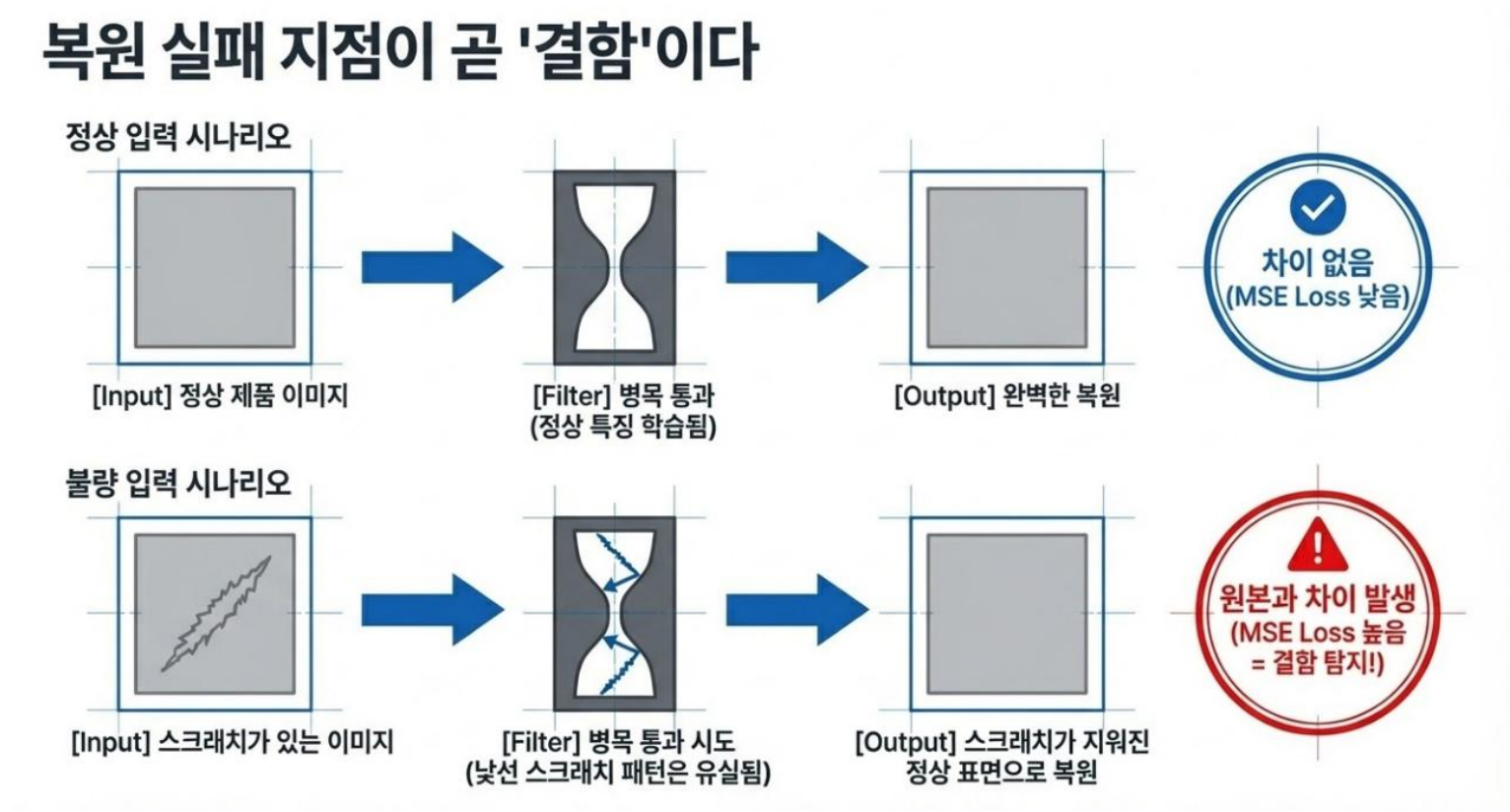
- 모델의 목적은 단순한 이미지 복사가 아님.
- 데이터를 억지로 좁은 병목(Bottleneck)에 통과시킴으로써, 가장 필수적인 정상 표면의 특징만 남기고 불필요한 노이즈는 버리는 법을 학습.

이미지 데이터를 활용한 AI 개발 – MVTec AD

INDUSTRIAL AI RESEARCH CENTER

■ 이미지 데이터를 활용한 AI 개발

➢ 합성곱 오토인코더(Convolutional Autoencoder, CAE)



■ 이미지 데이터를 활용한 AI 개발

➢ **합성곱 오토인코더(Convolutional Autoencoder, CAE)**

왜 최댓값(Max)인가? 제조 현장에서는 넓은 정상 표면 속에 아주 미세한 스크래치 하나만 있어도 불량(NG)으로 직결

픽셀 오차의 '평균'을 내버리면 이 미세한 불량 신호가 넓은 정상 면적에 희석

evaluate_performance() 함수는 전체 테스트 데이터셋을 순회하며 정상/불량 분류 능력을 정량적으로 평가합니다.

```
error_map = cv2.GaussianBlur(error_map, (15, 15), 0)

anomaly_score = np.max(error_map) # 최댓값 추출
```

수학적 스코어링의 핵심 지점

이미지 데이터를 활용한 AI 개발 – MVTec AD

INDUSTRIAL AI RESEARCH CENTER

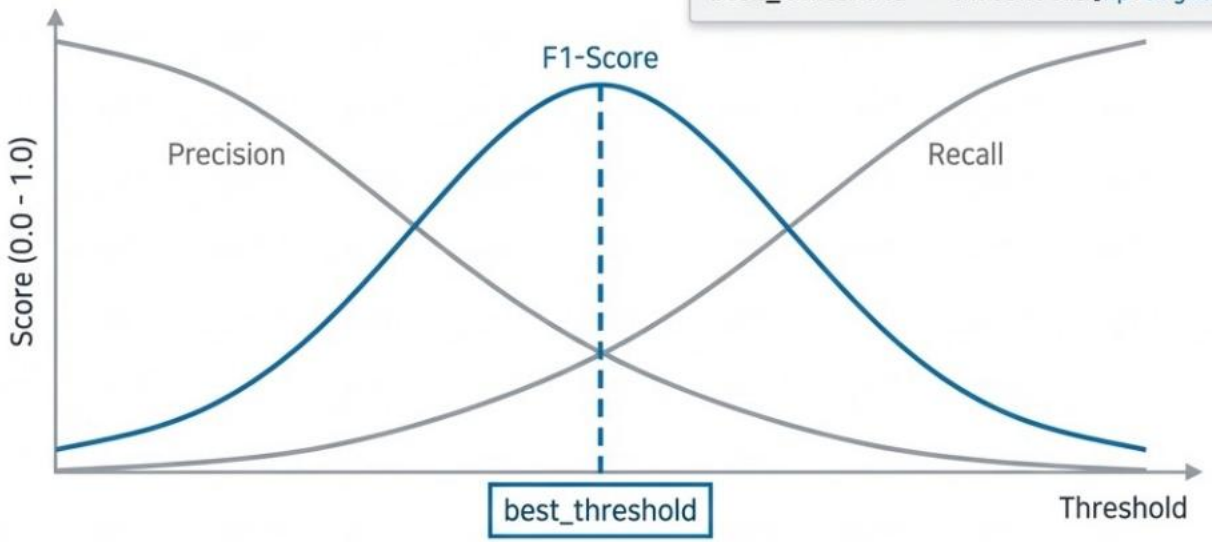
■ 이미지 데이터를 활용한 AI 개발

➢ **합성곱 오토인코더(Convolutional Autoencoder, CAE)**

알고리즘은 정상을 불량으로 오해하는 것과 불량을 놓치는 것 사이에서 수학적 밸런스가 가장 높은 지점(F1_socre 최대치)을 찾아냄

→ 해당 점수가 판정 기준이 됨

```
best_threshold = thresholds[np.argmax(f1_scores)]
```



이미지 데이터를 활용한 AI 개발 – MVTec AD

INDUSTRIAL AI RESEARCH CENTER

■ 이미지 데이터를 활용한 AI 개발

➢ **합성곱 오토인코더(Convolutional Autoencoder, CAE)**
수학적 오차를 사람이 인지할 수 있는 색상으로 매핑하여 원본 이미지 위에 overlay

```
cv2.normalize(..., err, 1)
```

```
cv2.applyColorMap(...,
    cv2.COLORMAP_JET)
```

```
cv2.addWeighted(...,
    cv2.finalWeights = 100)
```

감사합니다

Q&A